```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# Regression Models
from sklearn.linear_model import LinearRegression, Ridge
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor

# Classification Models
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import SVC

from sklearn.metrics import mean_squared_error, r2_score, accuracy_score, confusion_matrix
```

```
matches = pd.read_csv("/content/matches.csv")
deliveries = pd.read_csv("/content/deliveries.csv")
```

```
# Player stats
runs = deliveries.groupby('batter')['batsman_runs'].sum().reset_index()
balls = deliveries.groupby('batter')['ball'].count().reset_index().rename(columns={'ball': 'balls'})
matches_played = deliveries.groupby('batter')['match_id'].nunique().reset_index().rename(columns={'match_id': 'matches'

df = runs.merge(balls, on='batter').merge(matches_played, on='batter')

# Features
df['strike_rate'] = (df['batsman_runs'] / df['balls']) * 100
df['average'] = df['batsman_runs'] / df['matches']
```

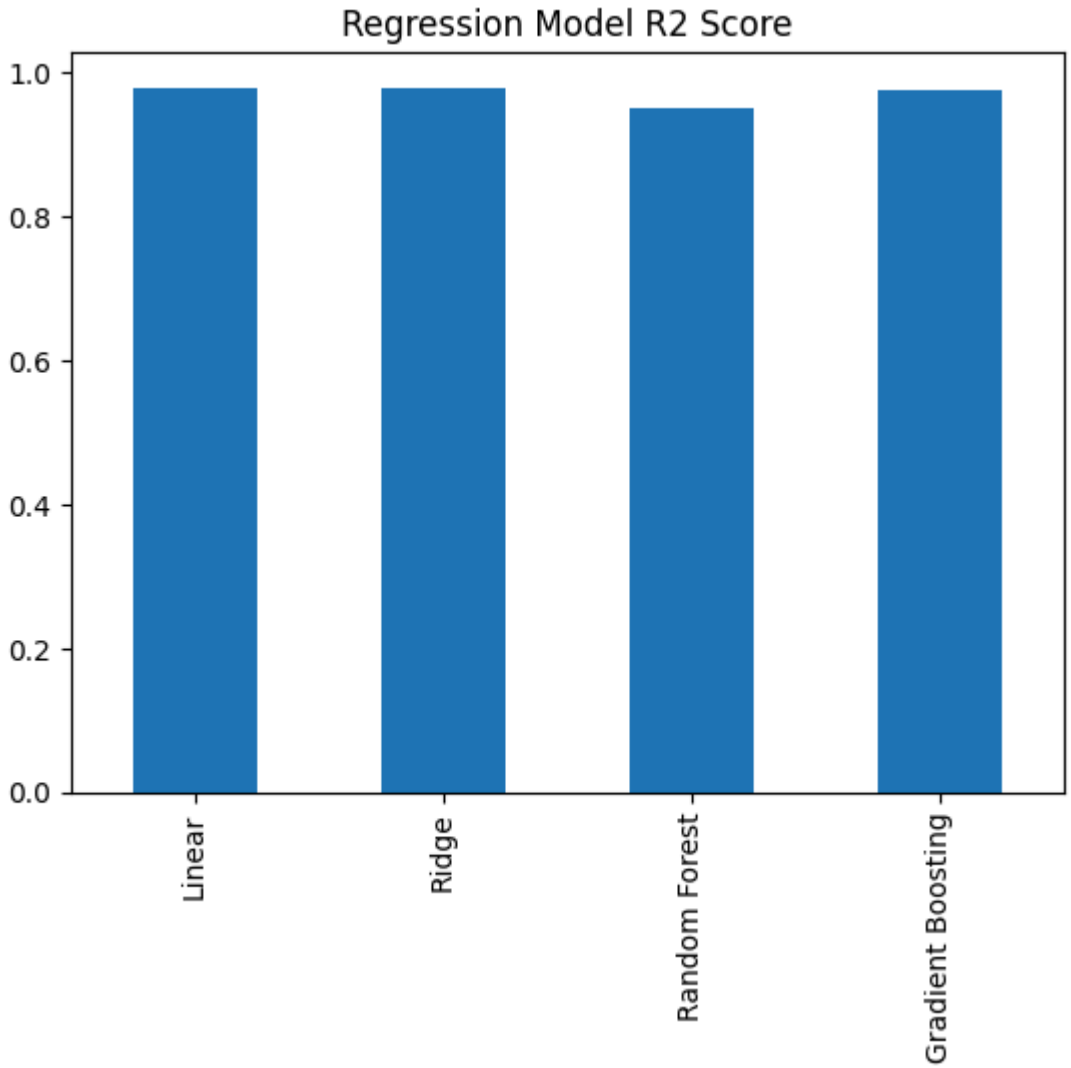
```
df = df[df['matches'] > 10]
df.dropna(inplace=True)
```

```
X_reg = df[['matches', 'balls', 'strike_rate', 'average']]
y_reg = df['batsman_runs']

X_train, X_test, y_train, y_test = train_test_split(X_reg, y_reg, test_size=0.2, random_state=42)
```

```
reg_models = {
    "Linear": LinearRegression(),
    "Ridge": Ridge(),
    "Random Forest": RandomForestRegressor(),
    "Gradient Boosting": GradientBoostingRegressor()
}

reg_results = {}

for name, model in reg_models.items():
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)

    reg_results[name] = {
        "MSE": mean_squared_error(y_test, y_pred),
        "R2": r2_score(y_test, y_pred)
    }

reg_df = pd.DataFrame(reg_results).T
print(reg_df)
```

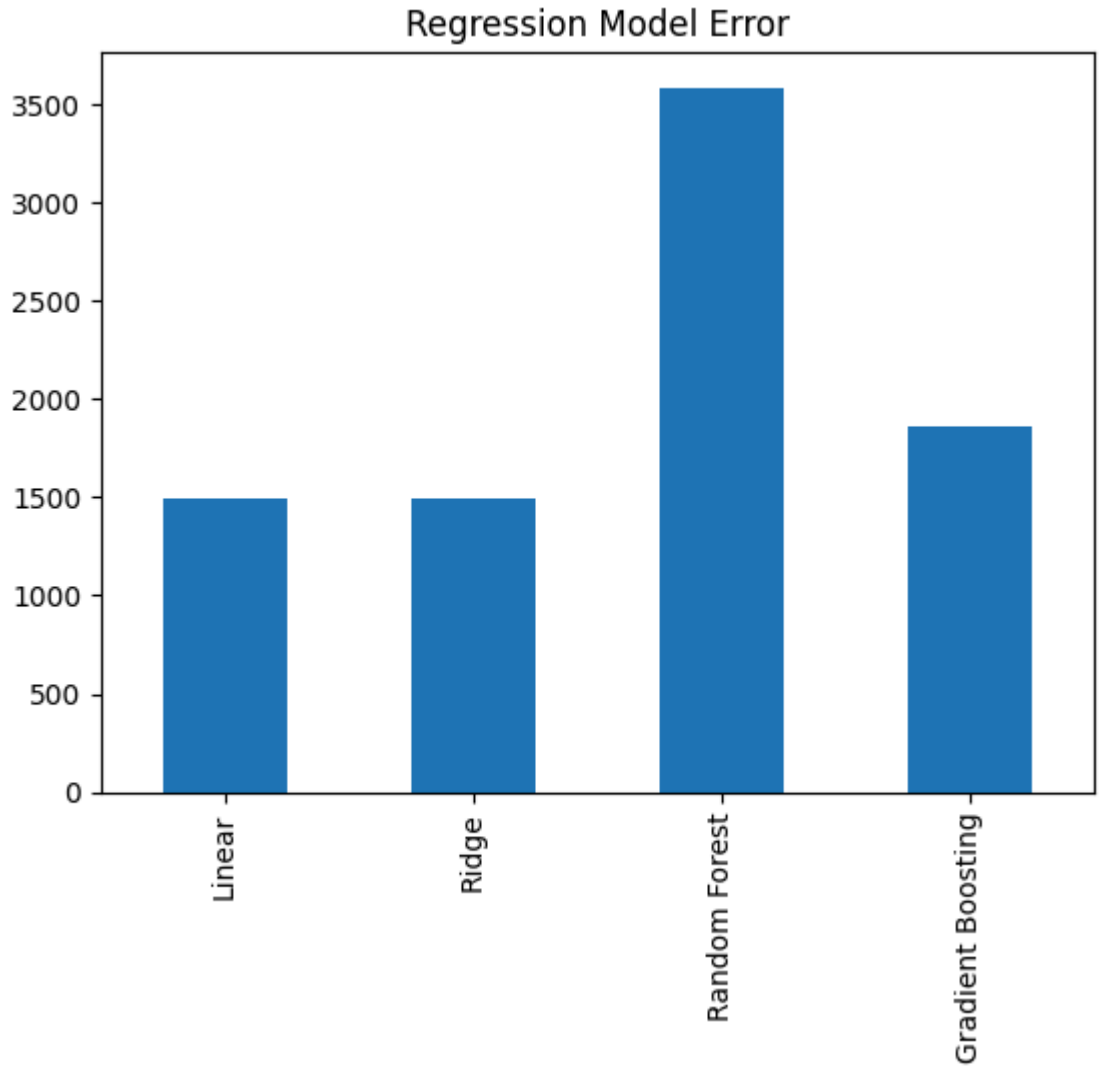
```
                          MSE        R2
Linear             1492.589746  0.980059
Ridge              1494.598994  0.980032
Random Forest      3590.175606  0.952035
Gradient Boosting  1860.316918  0.975146
```

```
reg_df['R2'].plot(kind='bar')
plt.title("Regression Model R2 Score")
plt.show()
```



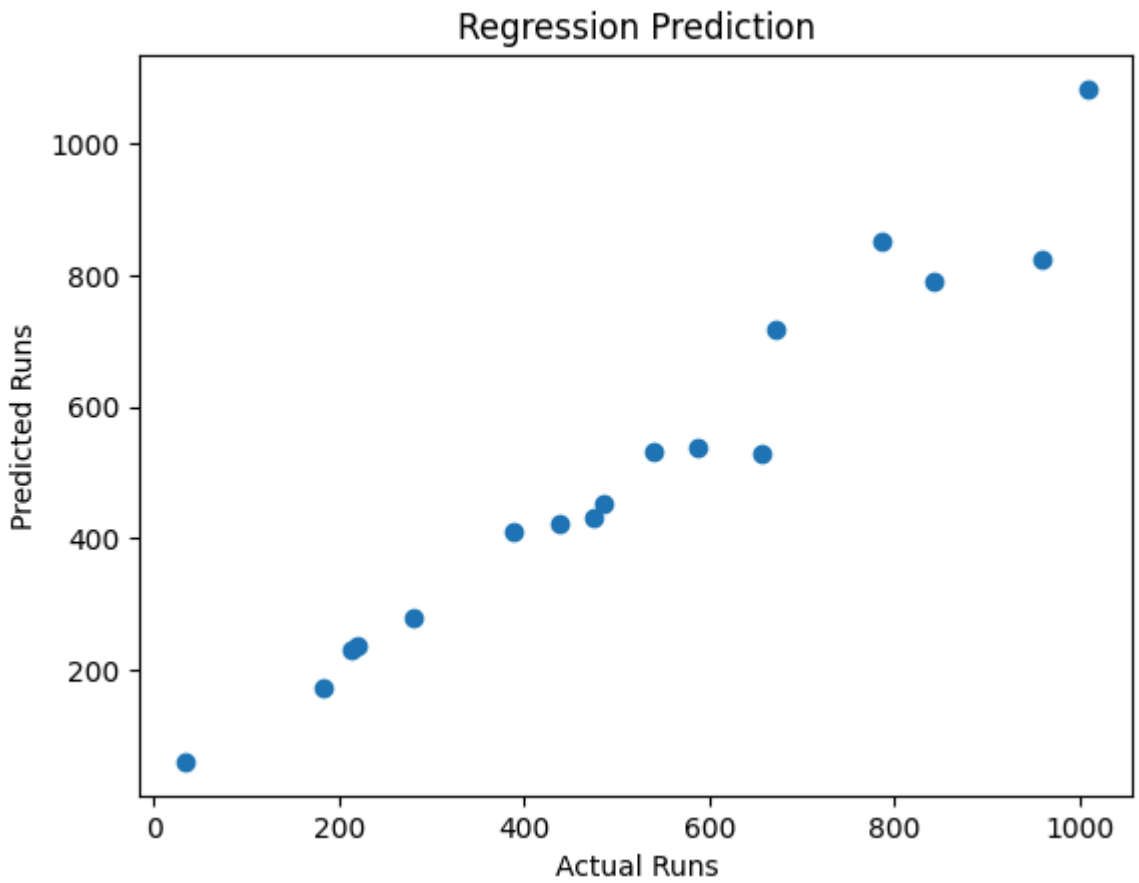
MSE COMPARISON

```
reg_df['MSE'].plot(kind='bar')
plt.title("Regression Model Error")
plt.show()
```



ACTUAL VS PREDICT

```
best_reg = RandomForestRegressor()
best_reg.fit(X_train, y_train)

y_pred = best_reg.predict(X_test)

plt.scatter(y_test, y_pred)
plt.xlabel("Actual Runs")
plt.ylabel("Predicted Runs")
plt.title("Regression Prediction")
plt.show()
```



Create Target Classes

```
# Create categories
df['performance'] = pd.cut(df['batsman_runs'],
                           bins=[0, 1000, 3000, 10000],
                           labels=['Low', 'Medium', 'High'])
```

Classification Data

```
X_clf = df[['matches', 'balls', 'strike_rate', 'average']]
y_clf = df['performance']

scaler = StandardScaler()
X_clf = scaler.fit_transform(X_clf)

X_train, X_test, y_train, y_test = train_test_split(X_clf, y_clf, test_size=0.2, random_state=42)
```

Train Classification Models

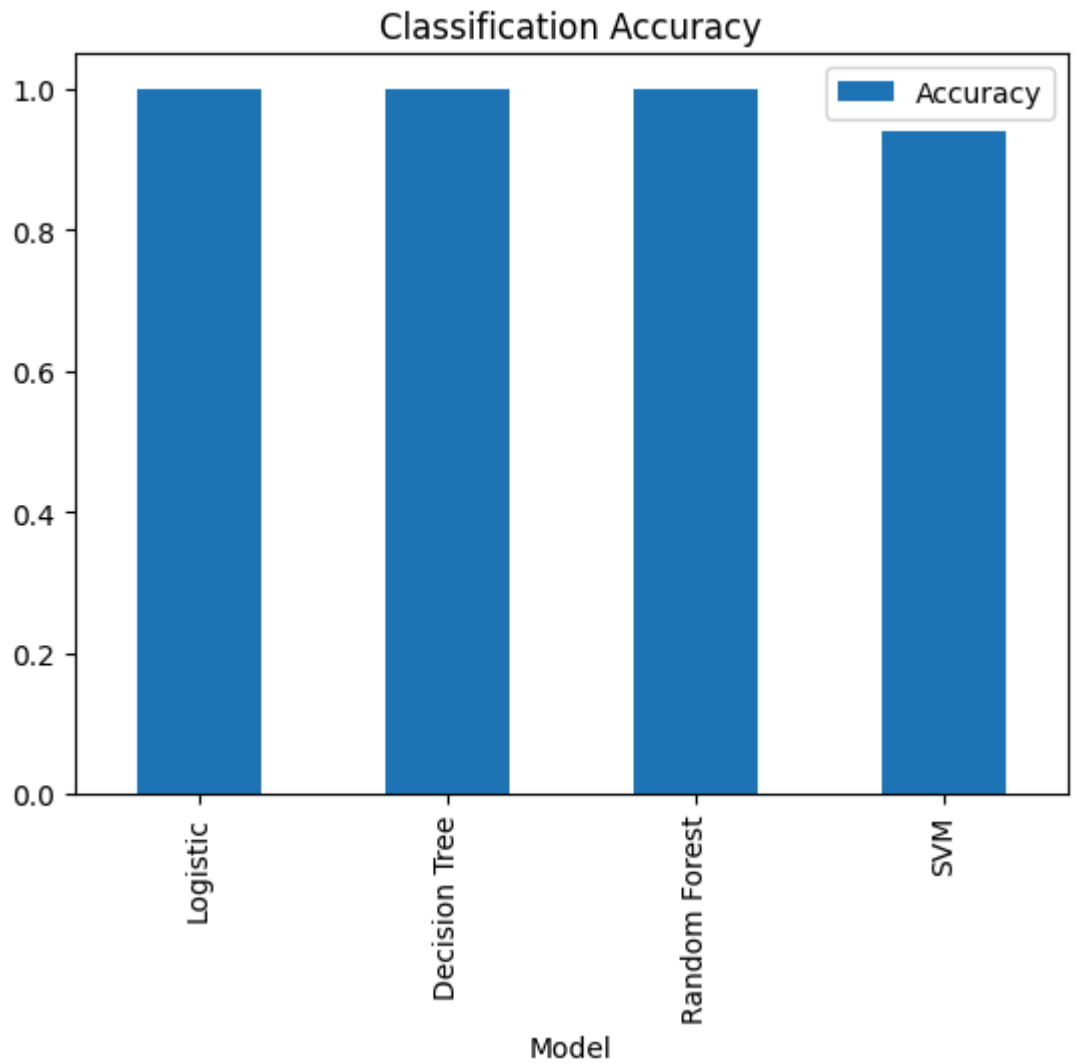
```
clf_models = {
    "Logistic": LogisticRegression(),
    "Decision Tree": DecisionTreeClassifier(),
    "Random Forest": RandomForestClassifier(),
    "SVM": SVC()
}

clf_results = {}

for name, model in clf_models.items():
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)

    clf_results[name] = accuracy_score(y_test, y_pred)
```

```
clf_df = pd.DataFrame(list(clf_results.items()), columns=["Model", "Accuracy"])
print(clf_df)
```

```
           Model  Accuracy
0       Logistic  1.000000
1  Decision Tree  1.000000
2  Random Forest  1.000000
3            SVM  0.941176
```

Accuracy Comparison

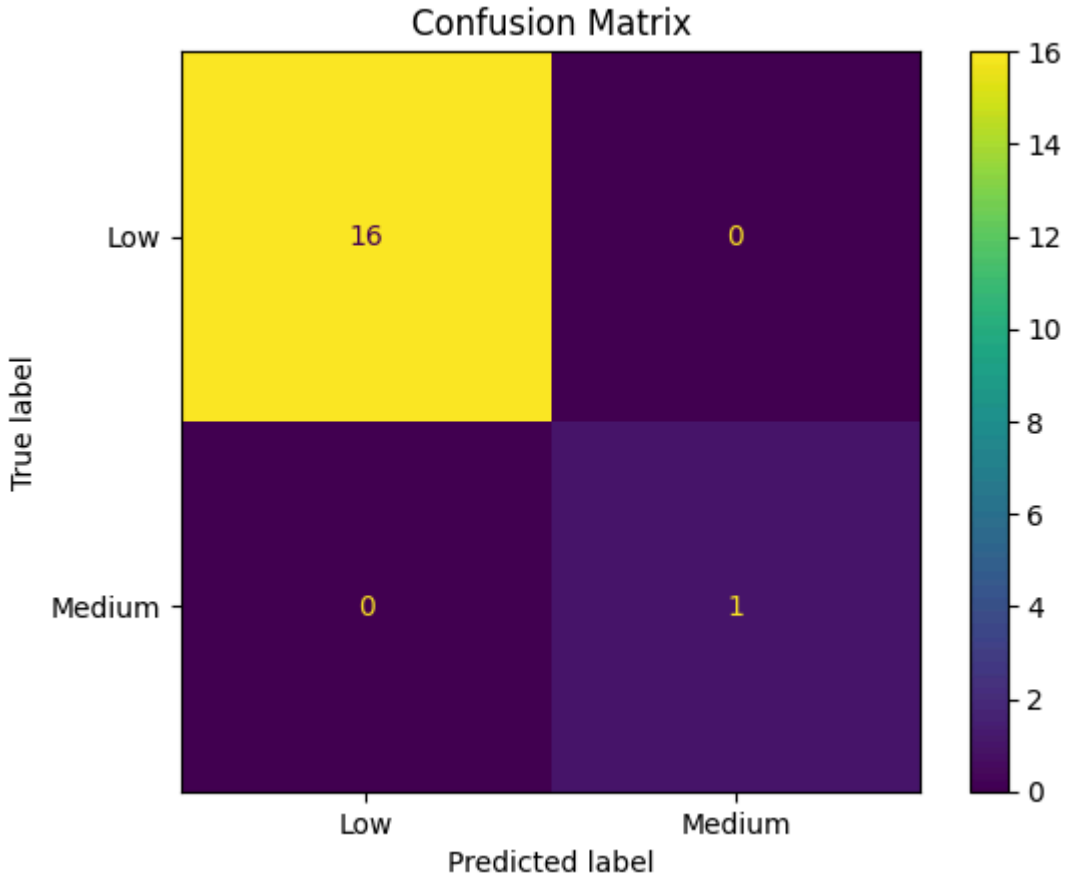
```
clf_df.set_index("Model").plot(kind='bar')
plt.title("Classification Accuracy")
plt.show()
```



Confusion Matrix

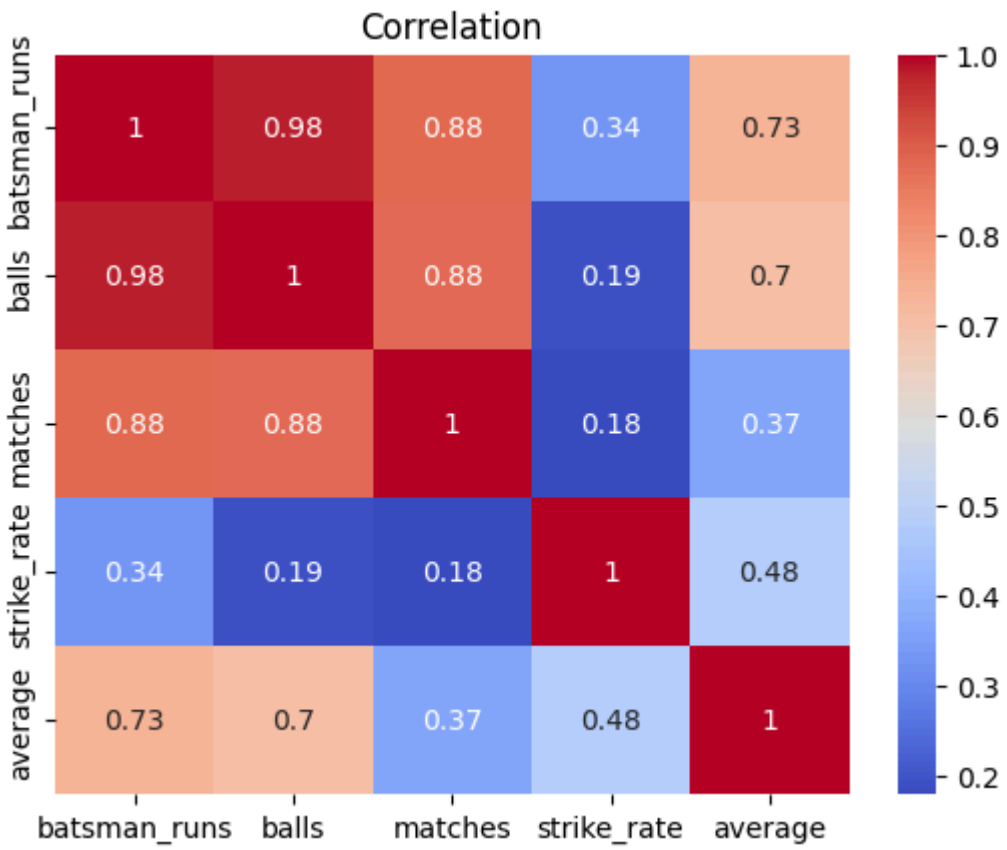
```
from sklearn.metrics import ConfusionMatrixDisplay

best_clf = RandomForestClassifier()
best_clf.fit(X_train, y_train)
```

```
ConfusionMatrixDisplay.from_estimator(best_clf, X_test, y_test)
plt.title("Confusion Matrix")
plt.show()
```



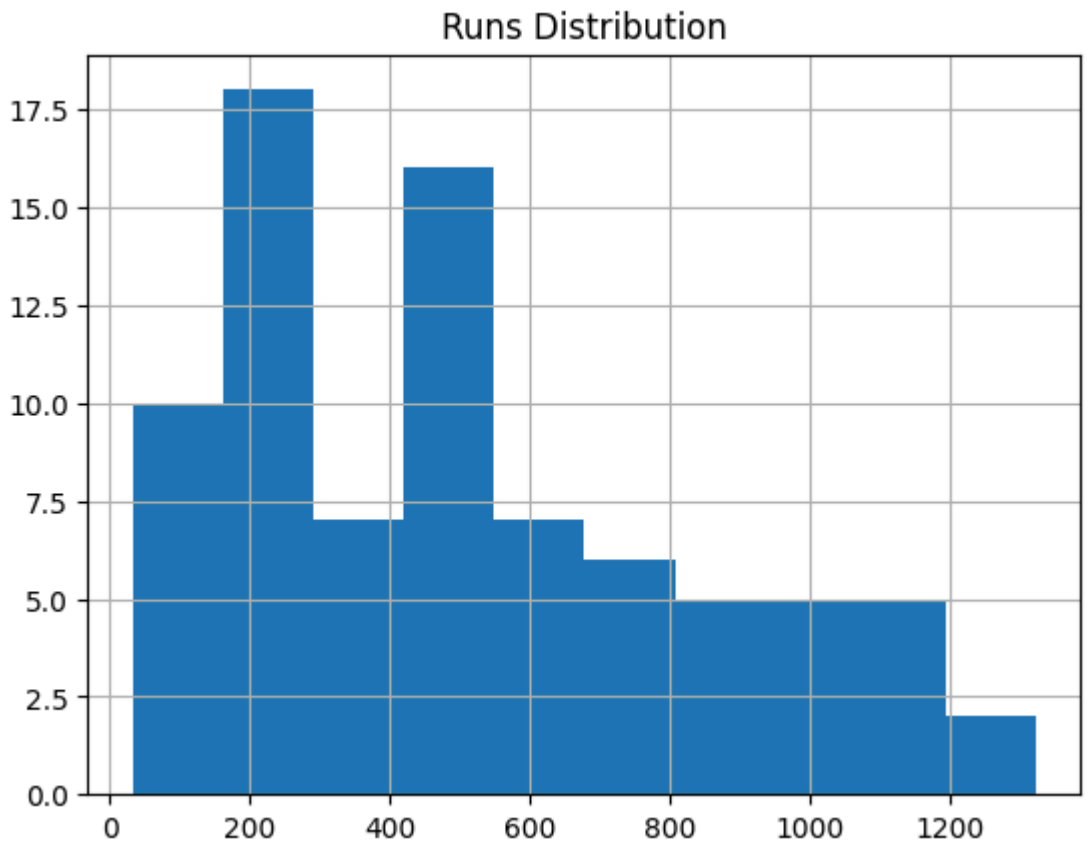
Correlation Heatmap

```
sns.heatmap(df.select_dtypes(include=np.number).corr(), annot=True, cmap='coolwarm')
plt.title("Correlation")
plt.show()
```



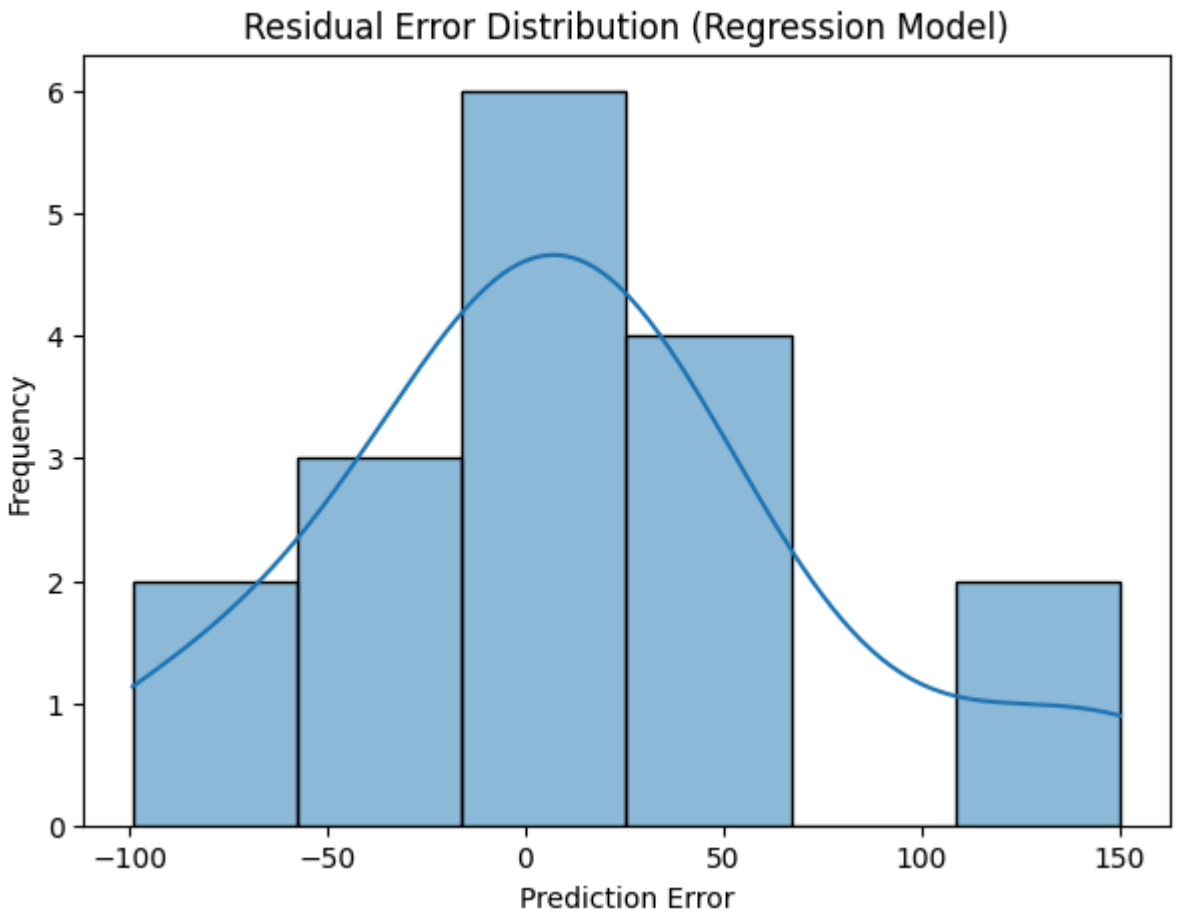
Runs Distribution

```
df['batsman_runs'].hist()
plt.title("Runs Distribution")
plt.show()
```



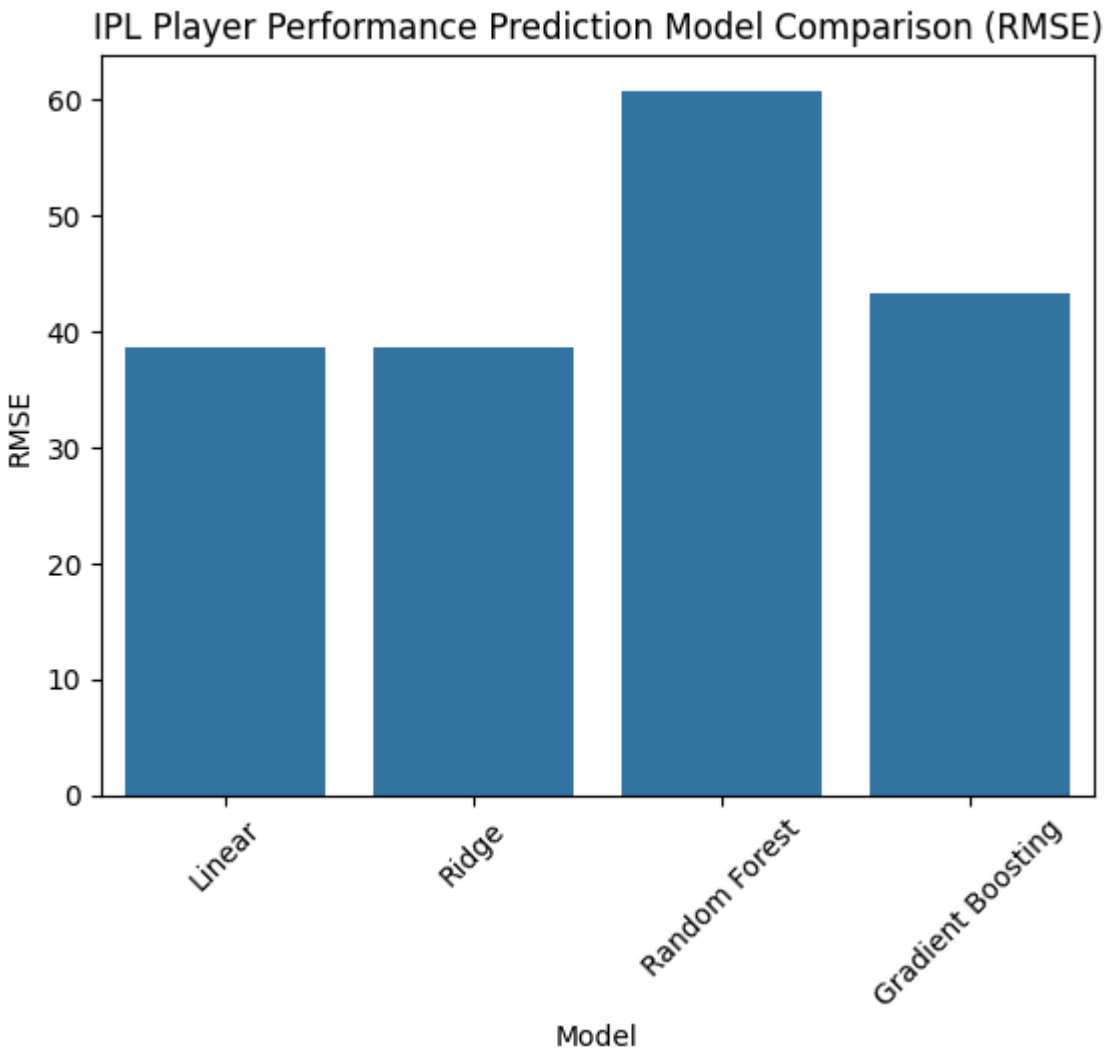

```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor

# Re-define X_reg and y_reg using the original DataFrame 'df'
X_reg_res = df[['matches', 'balls', 'strike_rate', 'average']]
y_reg_res = df['batsman_runs']

# Re-split data for regression to get consistent y_test and y_pred
X_train_reg_res, X_test_reg_res, y_train_reg_res, y_test_reg_res = train_test_split(X_reg_res, y_reg_res, test_size=0.2
```

```
# Re-instantiate and fit the best regression model (RandomForestRegressor) on the regression training data
best_reg_res = RandomForestRegressor(random_state=42) # Using random_state for reproducibility
best_reg_res.fit(X_train_reg_res, y_train_reg_res)

# Make predictions on the regression test data
y_pred_reg_res = best_reg_res.predict(X_test_reg_res)

# Calculate residuals
residuals = y_test_reg_res - y_pred_reg_res

plt.figure(figsize=(7,5))

sns.histplot(residuals, kde=True)

plt.title("Residual Error Distribution (Regression Model)")
plt.xlabel("Prediction Error")
plt.ylabel("Frequency")

plt.show()
```



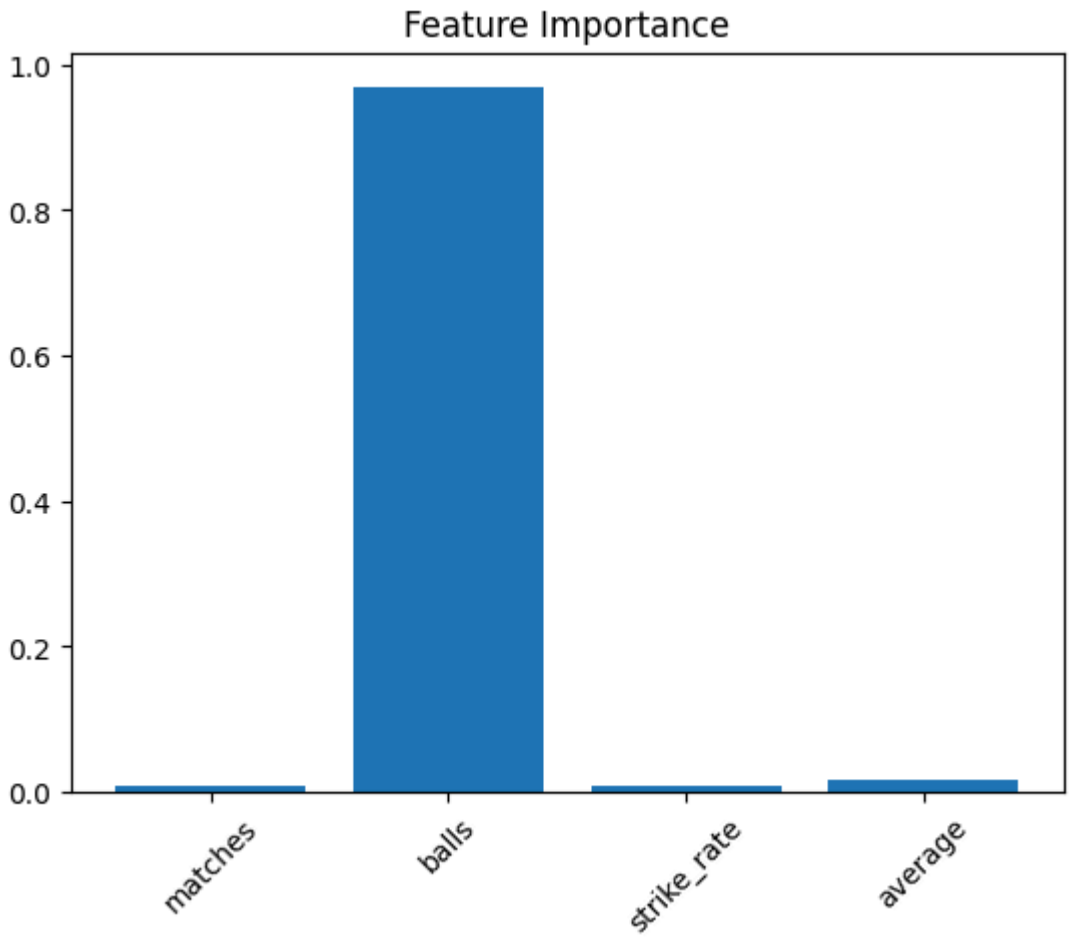

```
import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd
import numpy as np
from sklearn.metrics import mean_squared_error

results = []

for name, model in reg_models.items():
    # Use the regression-specific training and testing data
    model.fit(X_train_reg_res, y_train_reg_res)
```

```
    preds = model.predict(X_test_reg_res)

    rmse = np.sqrt(mean_squared_error(y_test_reg_res, preds))

    results.append([name, rmse])

results_df = pd.DataFrame(results, columns=['Model','RMSE'])

sns.barplot(x='Model', y='RMSE', data=results_df)

plt.title("IPL Player Performance Prediction Model Comparison (RMSE)")
plt.ylabel("RMSE")
plt.xticks(rotation=45)

plt.show()
```

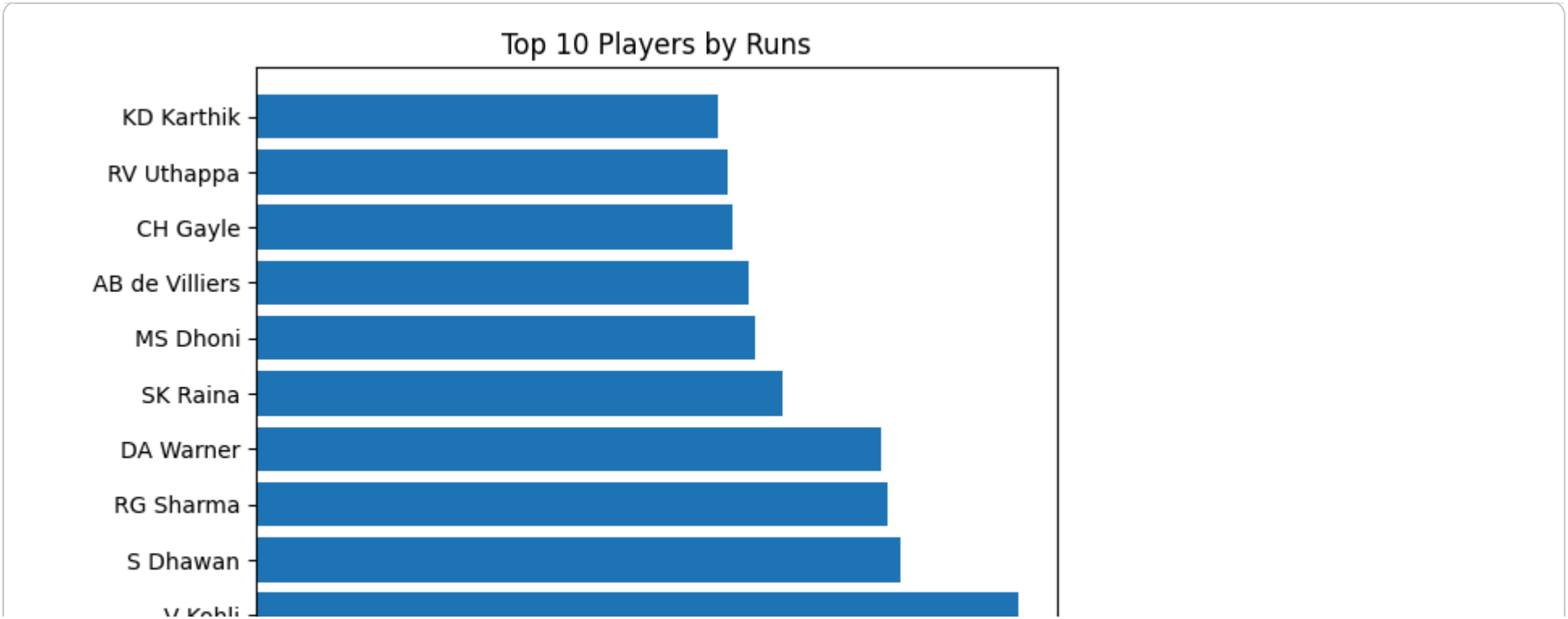



```
from sklearn.ensemble import RandomForestRegressor

rf = RandomForestRegressor()
rf.fit(X_train_reg_res, y_train_reg_res)

plt.bar(X_reg_res.columns, rf.feature_importances_)
plt.title("Feature Importance")
```

```
plt.xticks(rotation=45)
plt.show()
```




```
top_players = df.sort_values(by='batsman_runs', ascending=False).head(10)

plt.barh(top_players['batter'], top_players['batsman_runs'])
plt.title("Top 10 Players by Runs")
plt.xlabel("Runs")
plt.show()
```


Top 10 Players by Runs
KD Karthik
RV Uthappa
CH Gayle
AB de Villiers
MS Dhoni
SK Raina
DA Warner
RG Sharma
S Dhawan